```
      case (state: UninitializedState, command: Open) =>
        open(state, command, sharding, context, timer)
      case (state: OpenState, command: CheckMarketOdds) =>
        validateMarket(state, command)
      case (state: OpenState, command: RequestWalletFunds) =>
        validateFunds(state, command)
      case (state: OpenState, command: ValidationsTimedOut) =>
        checkValidations(state, command)
      case (state: OpenState, command: Settle) =>
        settle(state, command, sharding, context)
      case (state: OpenState, command: Close) =>
        finish(state, command)
      case (_, command: Cancel) => cancel(state, command)
      case (_, command: ReplyCommand)   => reject(state, command)
      case (_, command: Fail)           => fail(state, command)
      case _                            => invalid(state, command, context)
      }
    }
  }
  ...
}
```

Let's take a closer look at the `open` method. With the `Open` command in an `UninitializedState`, the bet starts a timer that schedules sending a message to the bet itself. Once this message arrives, all validations must be completed. The bet checks whether the market is available and the funds are reserved. After the timer is started, the `open` method generates and persists an `Opened` event. Then it initiates the two required validations: market and funds. Finally, it responds to the caller with `Accepted`.

Listing 12.8 Opening the bet

```
object Bet {
  ...

  private def open(
      state: UninitializedState,
      command: Open,
      sharding: ClusterSharding,
      context: ActorContext[Command],
      timers: TimerScheduler[Command]): ReplyEffect[Opened, State] = {
    timers.startSingleTimer(                                          ❶
```